Project Identification and Diagram Selection

Blueprinting exercise response

Proposal title

SmartCivi Civil Registry Request and Appointment Management System

SmartCivi is a web-based system for citizens to register, sign in, submit civil registry document requests, track request status, and record payment details. Administrators review requests, update statuses, and manage user accounts, while appointment and supporting-document models provide a foundation for further development.

Structural selection System Architecture Diagram

This diagram is essential because it identifies the browser, Express application, MySQL database, shared Docker network, and persistent volume. It gives the developer the infrastructure boundaries and connection paths needed to run the actual system.

Behavioral selection Use Case Diagram

The Citizen actor registers, signs in, submits a request, views personal requests, and records a payment for an approved request. The Administrator actor reviews requests, changes statuses, and manages accounts. This establishes the system scope and separates the two roles.

Behavioral selection Sequence Diagram

The selected scenario is an authenticated citizen submitting a civil registry request. Messages proceed from the browser to the router, request controller, Sequelize model, and MySQL; the controller then redirects the citizen to a confirmation page.

Data Blueprinting and Diagram Guide

Exercise response and reading key

How data flows to the database

A citizen enters a full name, email, phone number, document type, and purpose in the request form. The browser posts these values to POST /request. The Express controller verifies the session and required fields, then Request.create sends an SQL insert through Sequelize and mysql2 to the civil_requests table in MySQL.

The saved row receives an ID and the status pending. The controller redirects the browser to /request/{id}/confirmation, where the record is read and rendered as a confirmation page. My Requests locates personal records by the signed-in email. Administrators read and update these records through the review interface.

The database writes its files under /var/lib/mysql in the smartcivi_mysql_data volume. Application sessions remain in memory, so database records survive container recreation while existing sign-ins do not.

Contents

Structural diagrams: Architecture, Class, Component, Deployment, Object, Package, Composite Structure, and Profile.

Behavioral reference diagrams: DFD, Use Case, Activity, Sequence, State Machine, Communication, Interaction Overview, Timing, and Requirement. The DFD includes separate context and Level One views, giving 18 diagram figures across 17 diagram types.

Reading key

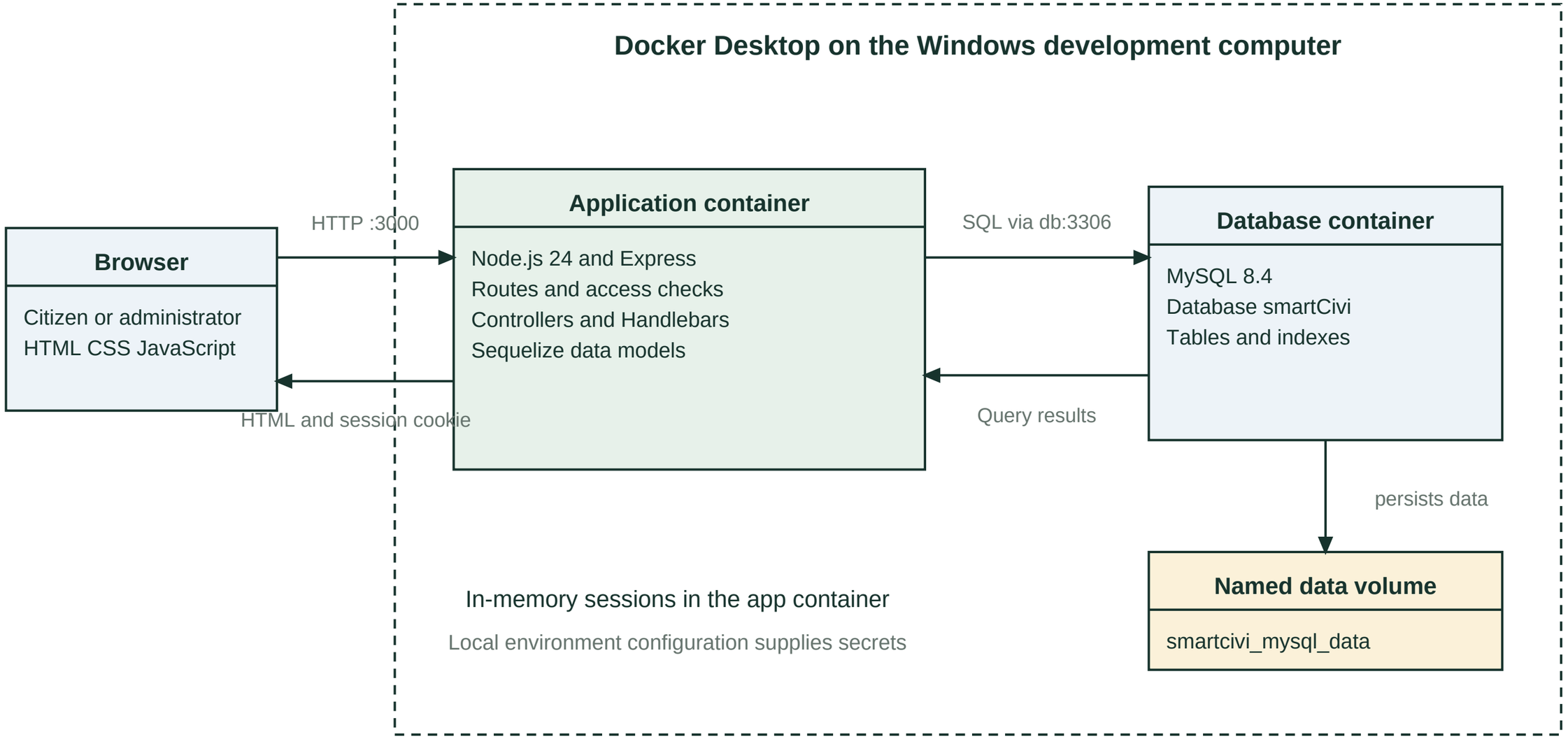
Arrows show direction; labels identify data, messages, dependencies, or transitions according to the diagram type. Class relationship ends show multiplicities. Gold boxes identify supporting records, requirements, or alternative outcomes. Profile definitions and object instances are explicitly illustrative.

The three exercise selections are marked CORE SELECTION in their page headings. Supplementary diagrams extend the reference coverage and are not additional minimum requirements of the exercise.

System Architecture

Structural diagram | CORE SELECTION

Show the deployed application layers and their connections.

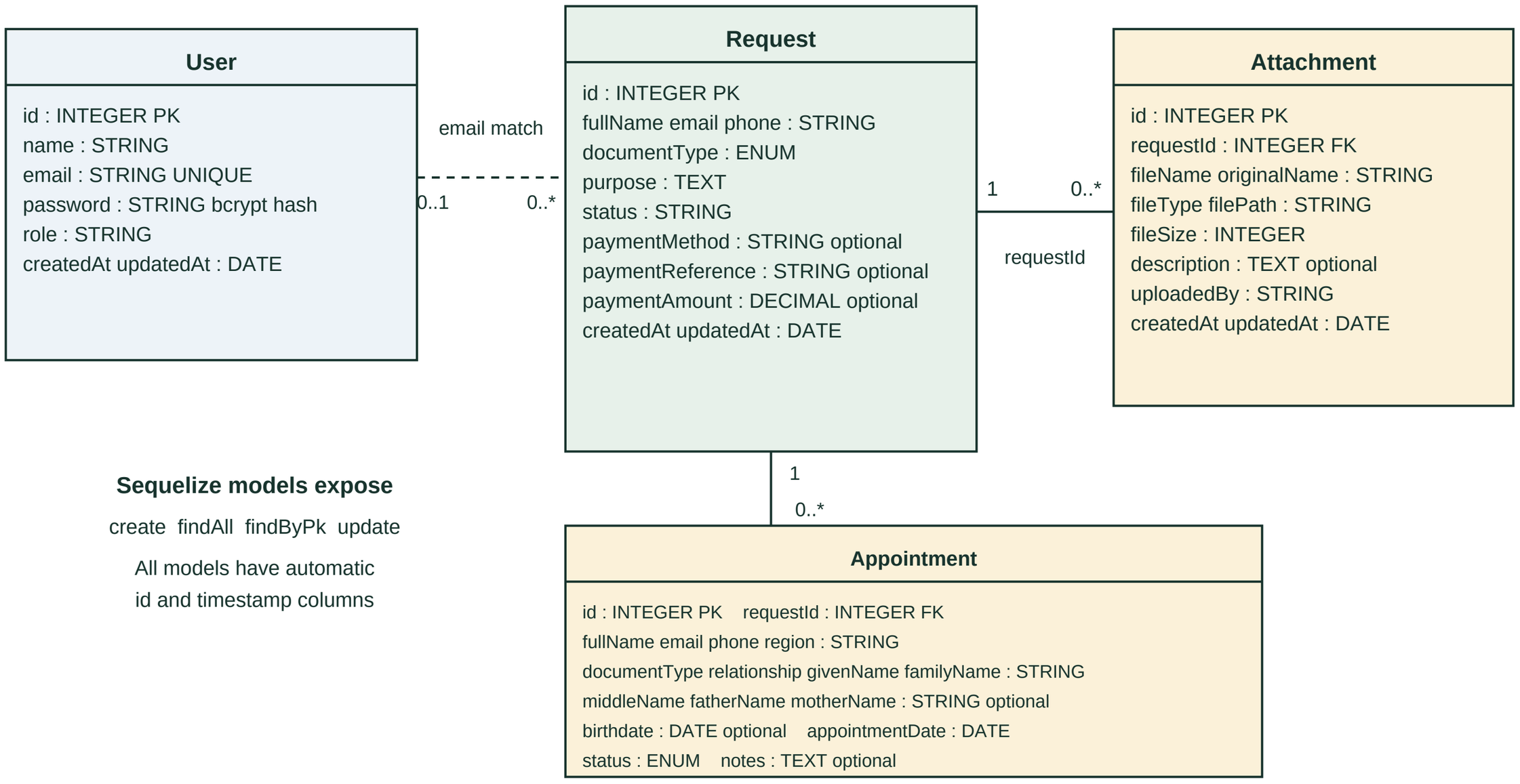


The browser receives server-rendered Handlebars pages. React, Nginx and a cloud service are not part of this deployment.

Class Diagram

Structural diagram | Supplementary reference view

Describe the four database models and their actual relationships.



Sequelize models expose

create findAll findByPk update

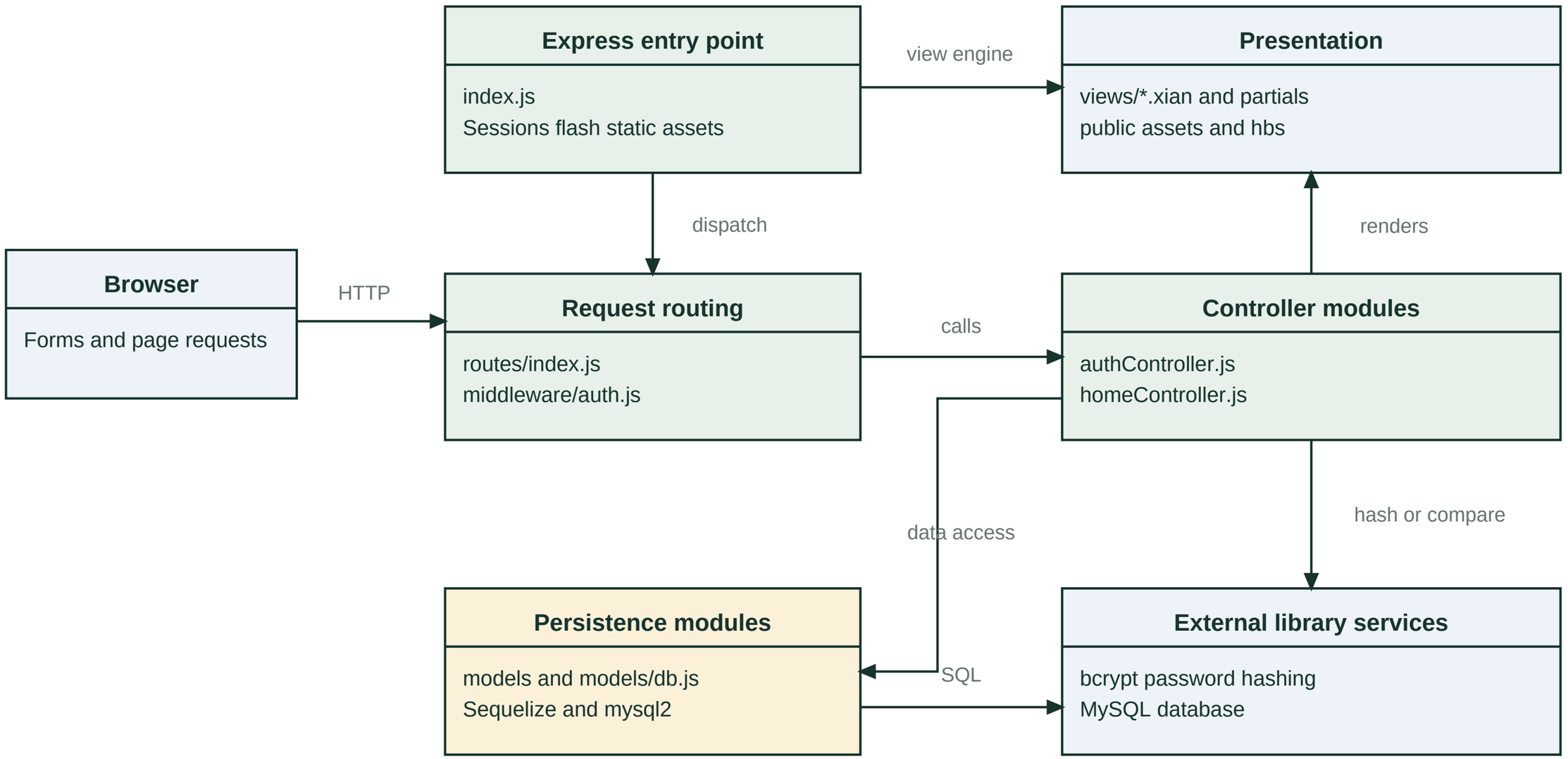
All models have automatic id and timestamp columns

User to Request is an email-based application association, not a database foreign key. Appointment and Attachment are defined models with incomplete feature wiring.

Component Diagram

Structural diagram | Supplementary reference view

Show software modules and library dependencies.

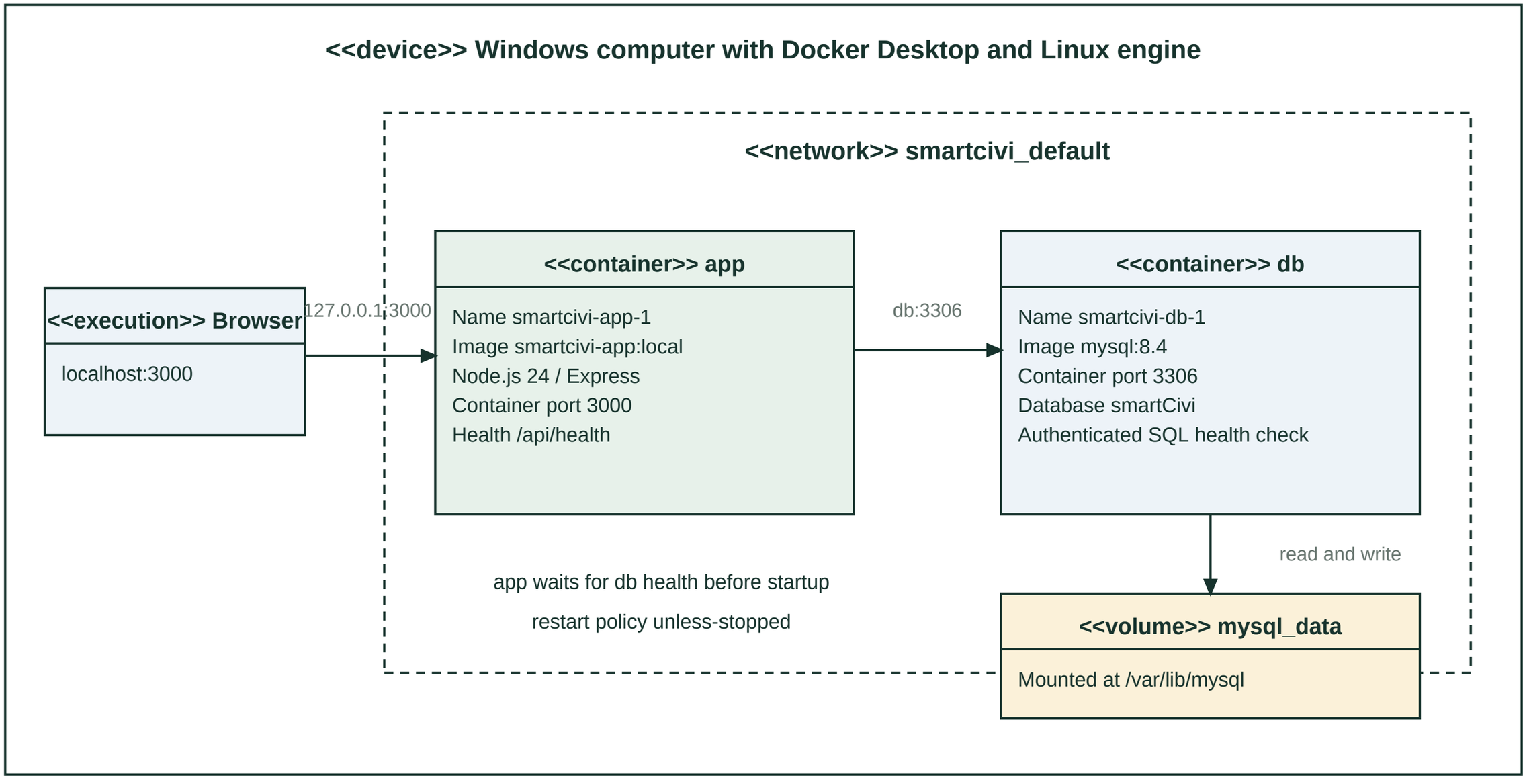


Controllers contain both business logic and page rendering. This is one Express application rather than separate frontend and backend services.

Deployment Diagram

Structural diagram | Supplementary reference view

Map runtime artifacts to Docker containers, network and storage.

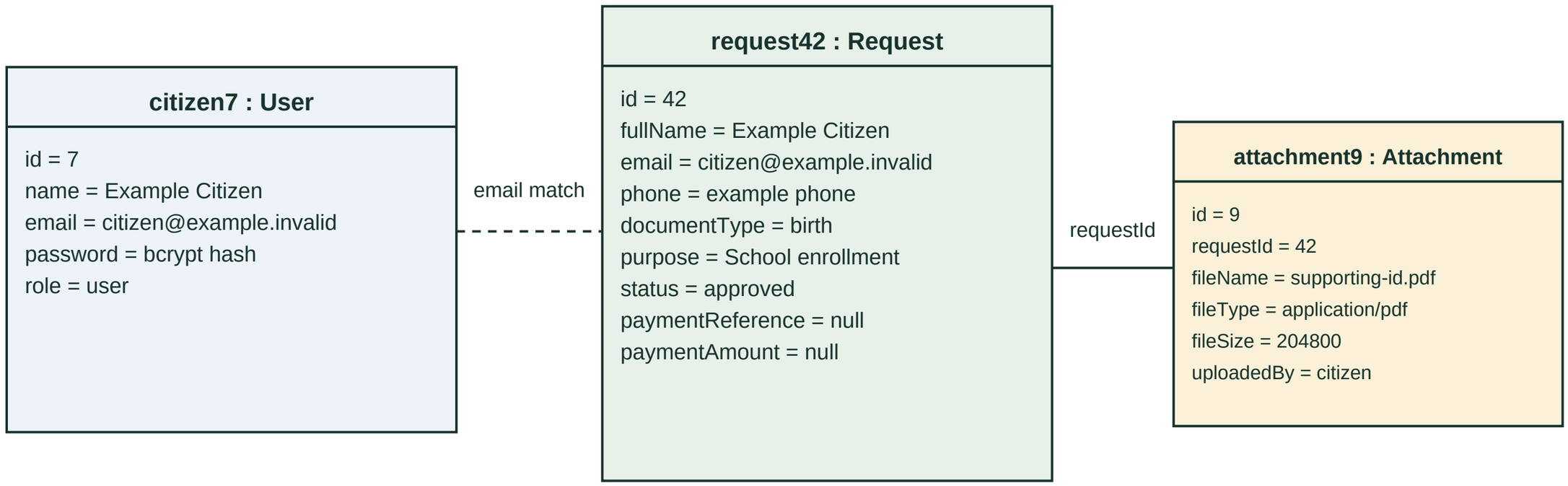


The app is bound to the host loopback address. MySQL has no published host port. Database storage survives container recreation.

Object Diagram

Structural diagram | Supplementary reference view

Illustrate one possible data snapshot with object instances.



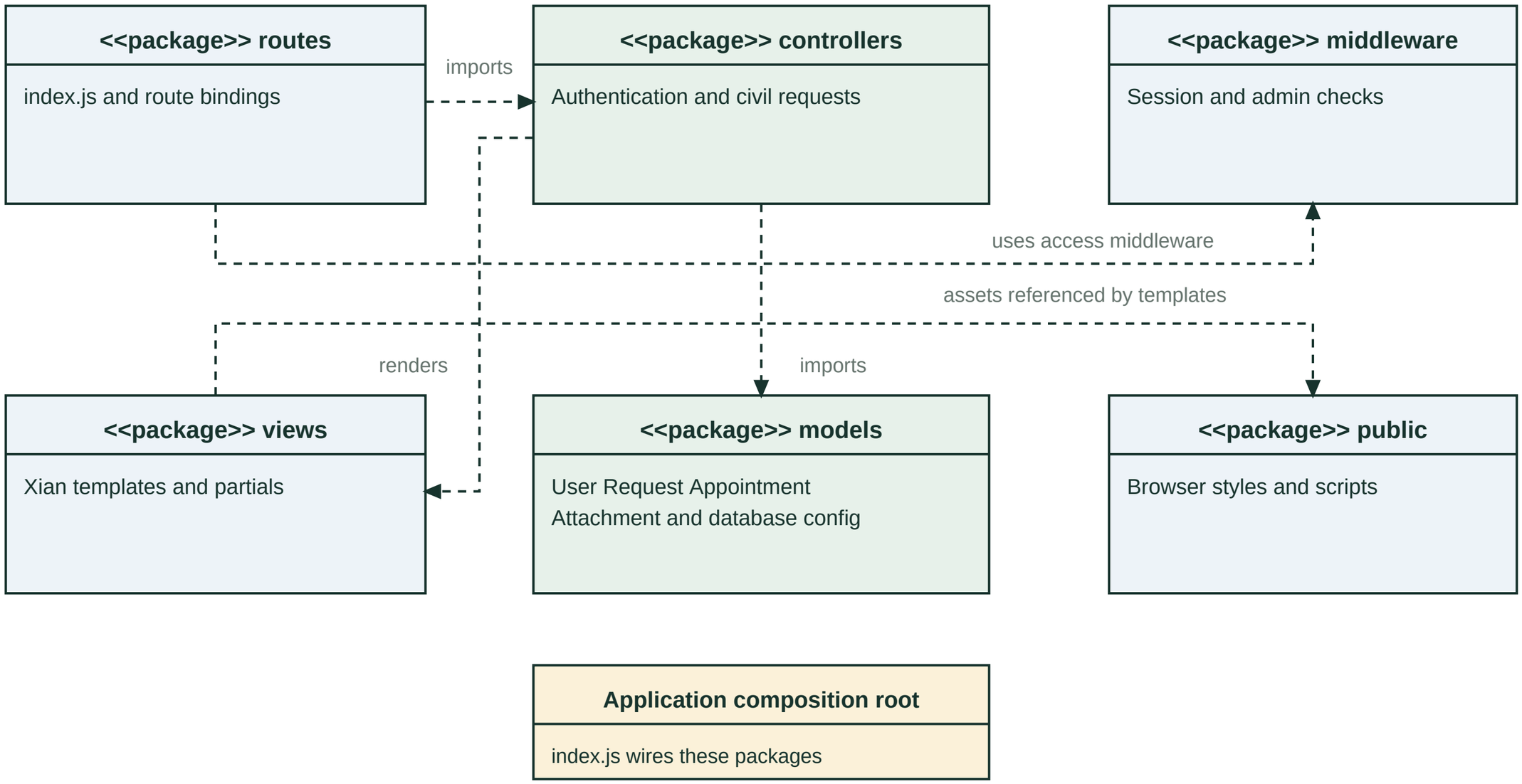
Snapshot only: example instances do not represent live records

All names, IDs and values on this page are fictional examples. A request owner is located by matching email; it has no userId column.

Package Diagram

Structural diagram | Supplementary reference view

Group repository modules into their existing folders.

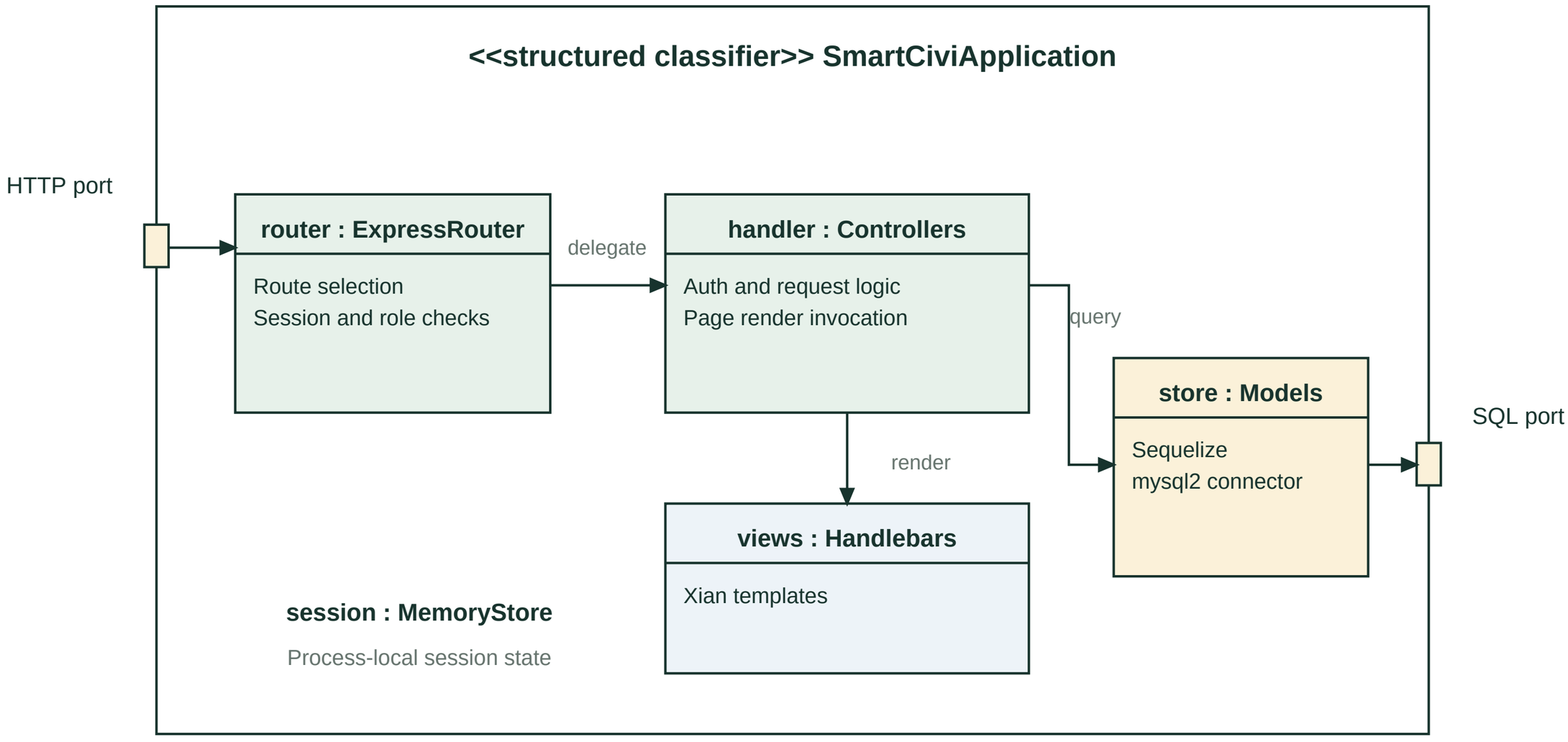


Arrows indicate source dependencies. The config folder includes a mock collection interface; the active relational connection is models/db.js.

Composite Structure Diagram

Structural diagram | Supplementary reference view

Expand the internal parts of the Express application classifier.

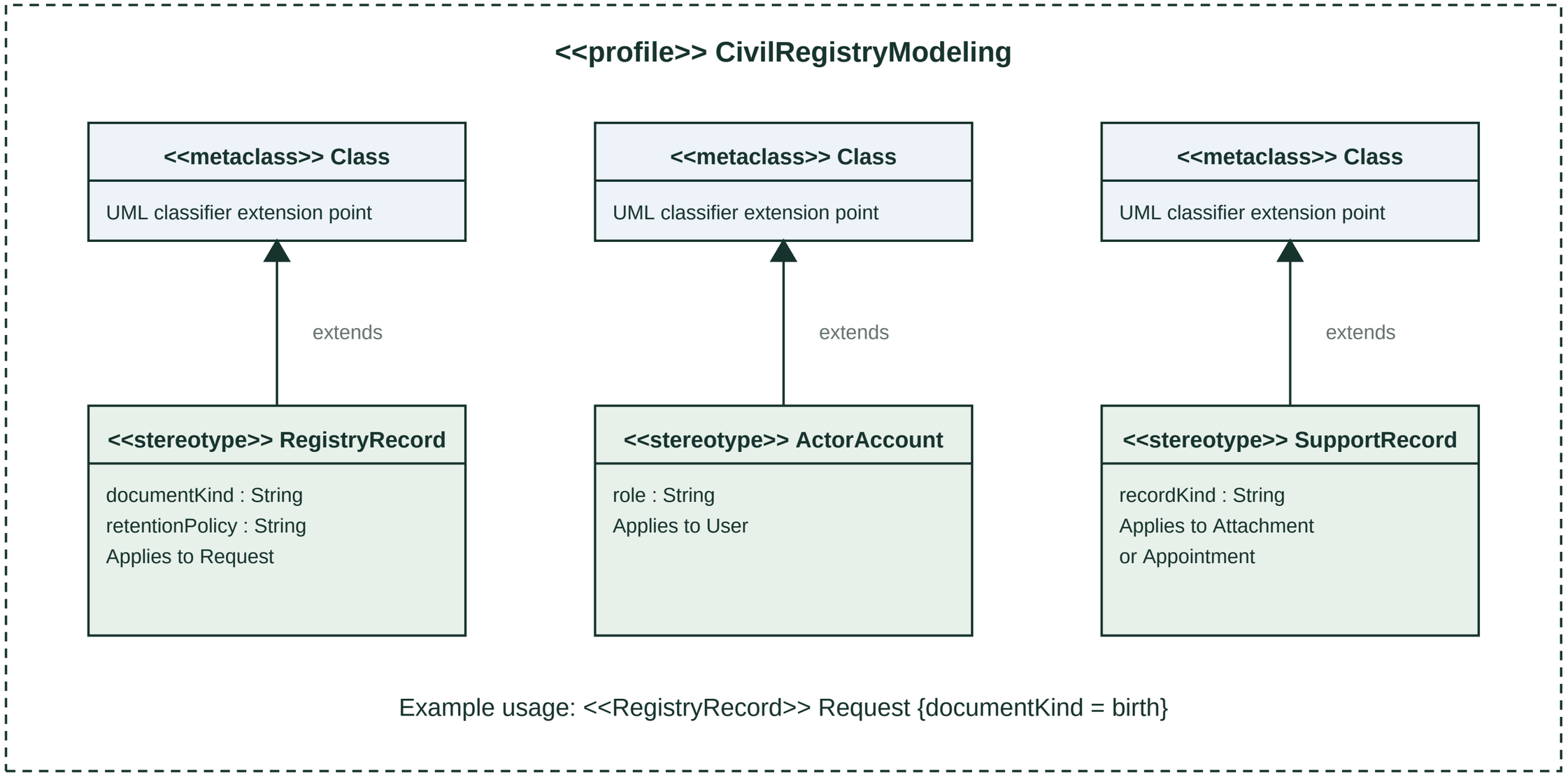


Internal parts represent existing module responsibilities. The connection to MySQL is a required external interface, not an internal database copy.

Profile Diagram

Structural diagram | Supplementary reference view

Demonstrate optional UML vocabulary for the civil registry domain.

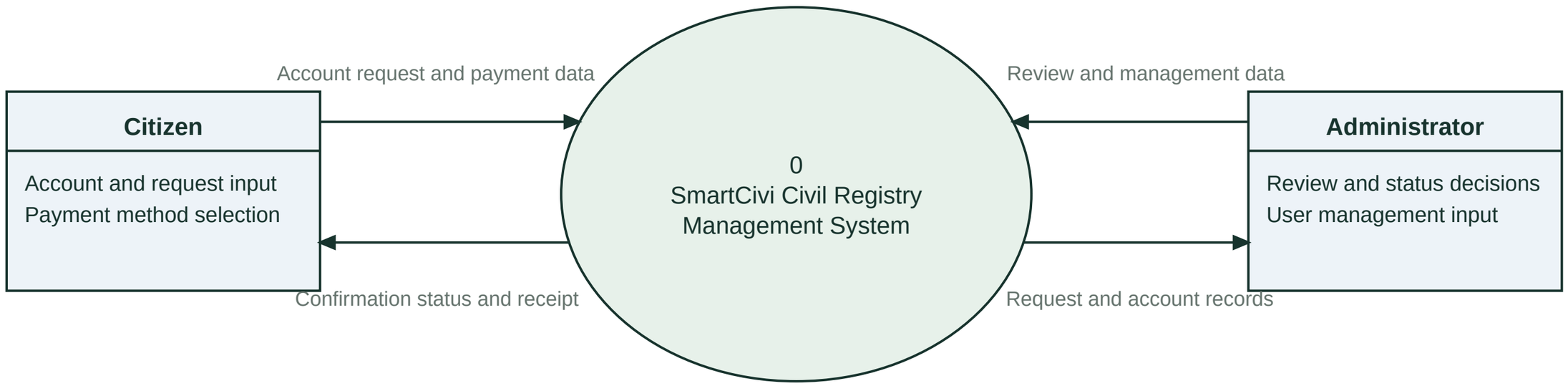


Proposed documentation profile only. SmartCivi does not implement or require a custom UML profile. Tagged values describe model elements, not runtime behavior.

Data Flow Diagram Context

Behavioral diagram | Supplementary reference view

Define the system boundary and external data exchanges.



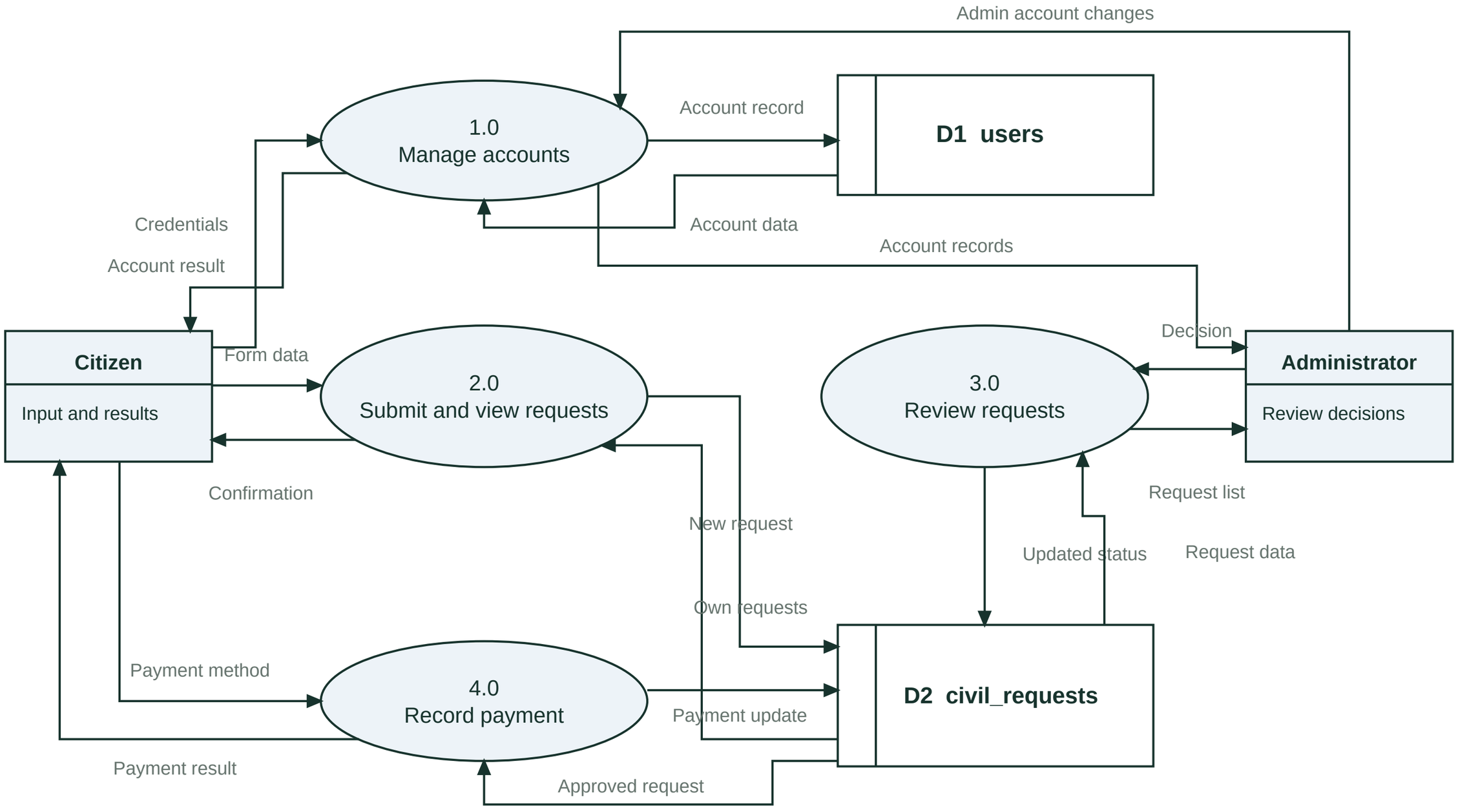
External entities exchange data only through process 0

Context-level DFD. Payment details stay inside SmartCivi in this prototype; no external payment gateway is connected.

Data Flow Diagram Level One

Behavioral diagram | Supplementary reference view

Trace account, request, review and payment data to persistent stores.

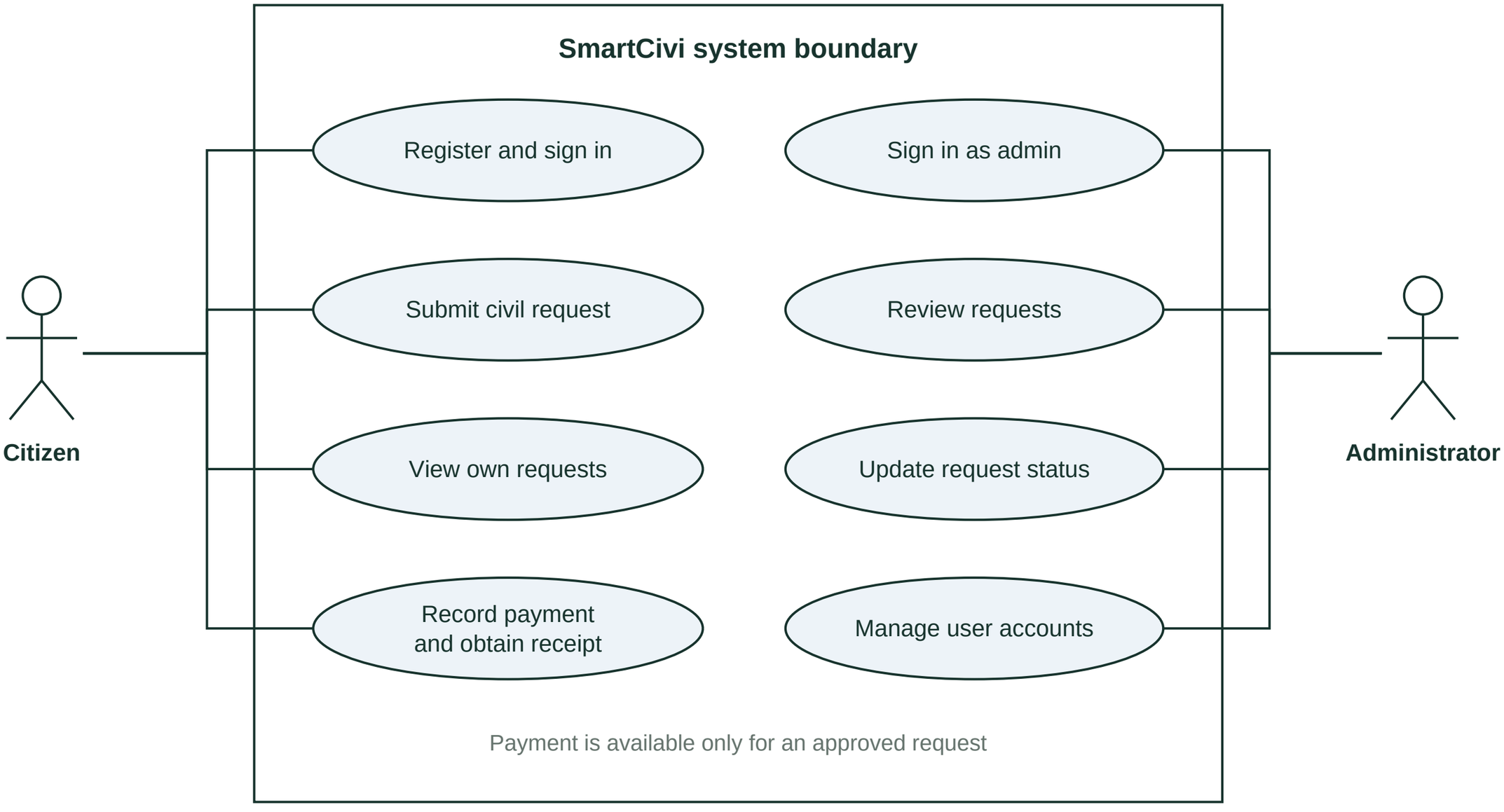


D1 and D2 are MySQL tables. Appointment and attachment stores are omitted here because these end-to-end flows are not fully connected in the controllers.

Use Case Diagram

Behavioral diagram | CORE SELECTION

Map external actors to the main system goals.

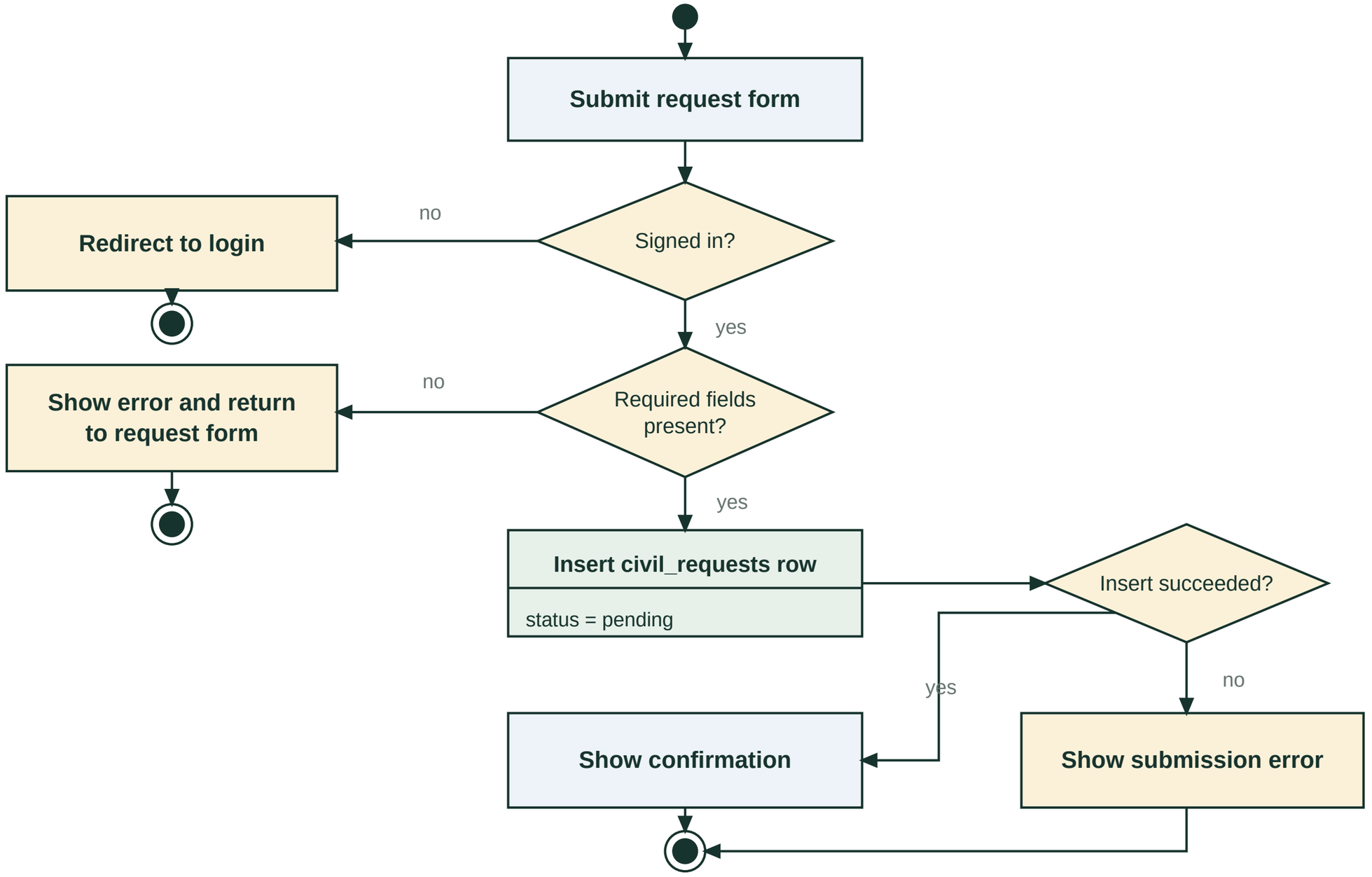


Core use cases reflect existing account and request controllers. Appointment scheduling, uploads and external payment processing remain supplementary design goals.

Activity Diagram

Behavioral diagram | Supplementary reference view

Follow the request submission workflow including validation and storage failures.

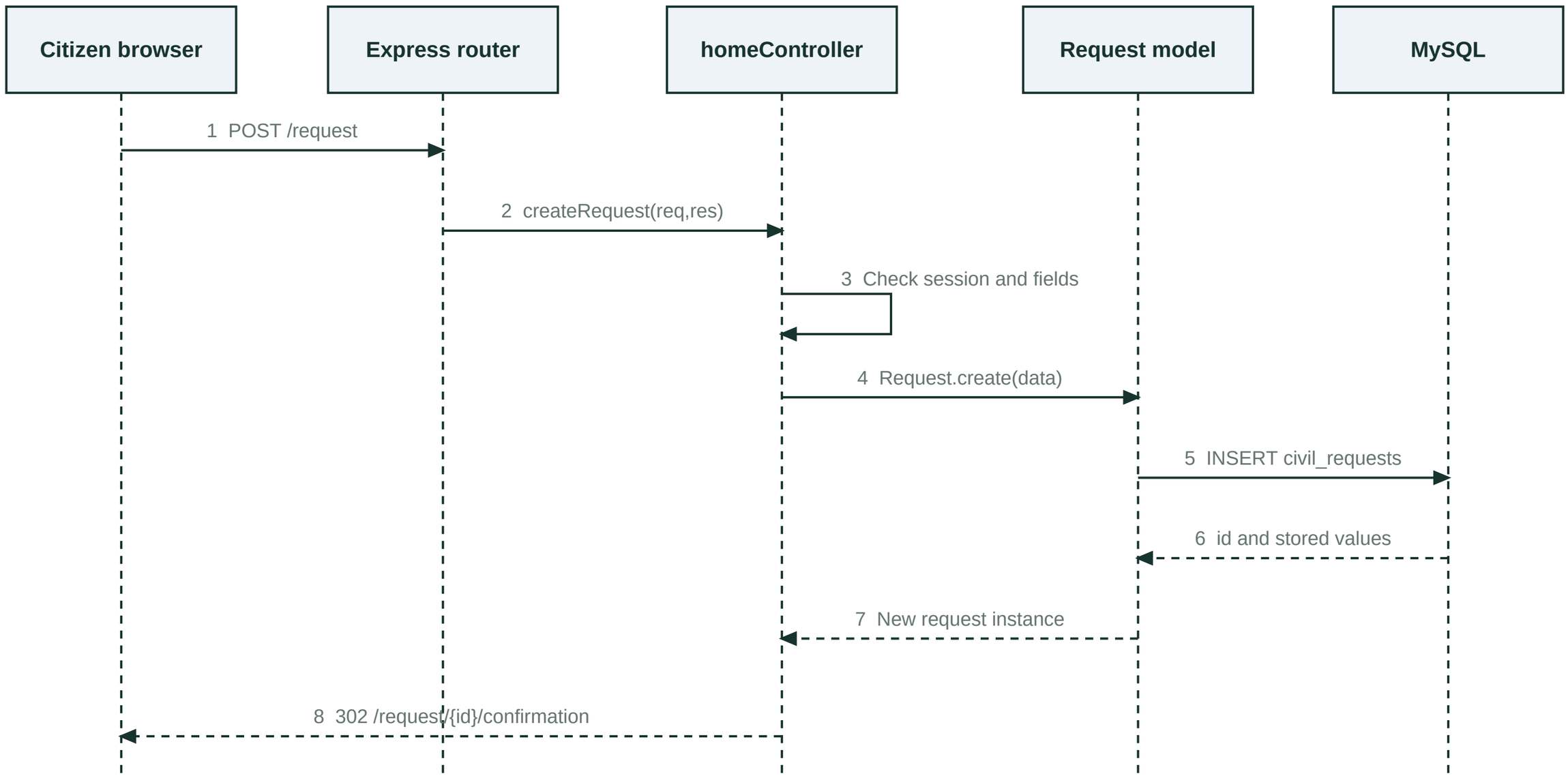


This activity describes POST /request and the subsequent confirmation page. It does not claim that administrator review occurs during submission.

Sequence Diagram

Behavioral diagram | CORE SELECTION

Show chronological request submission from browser to MySQL and the confirmation redirect.

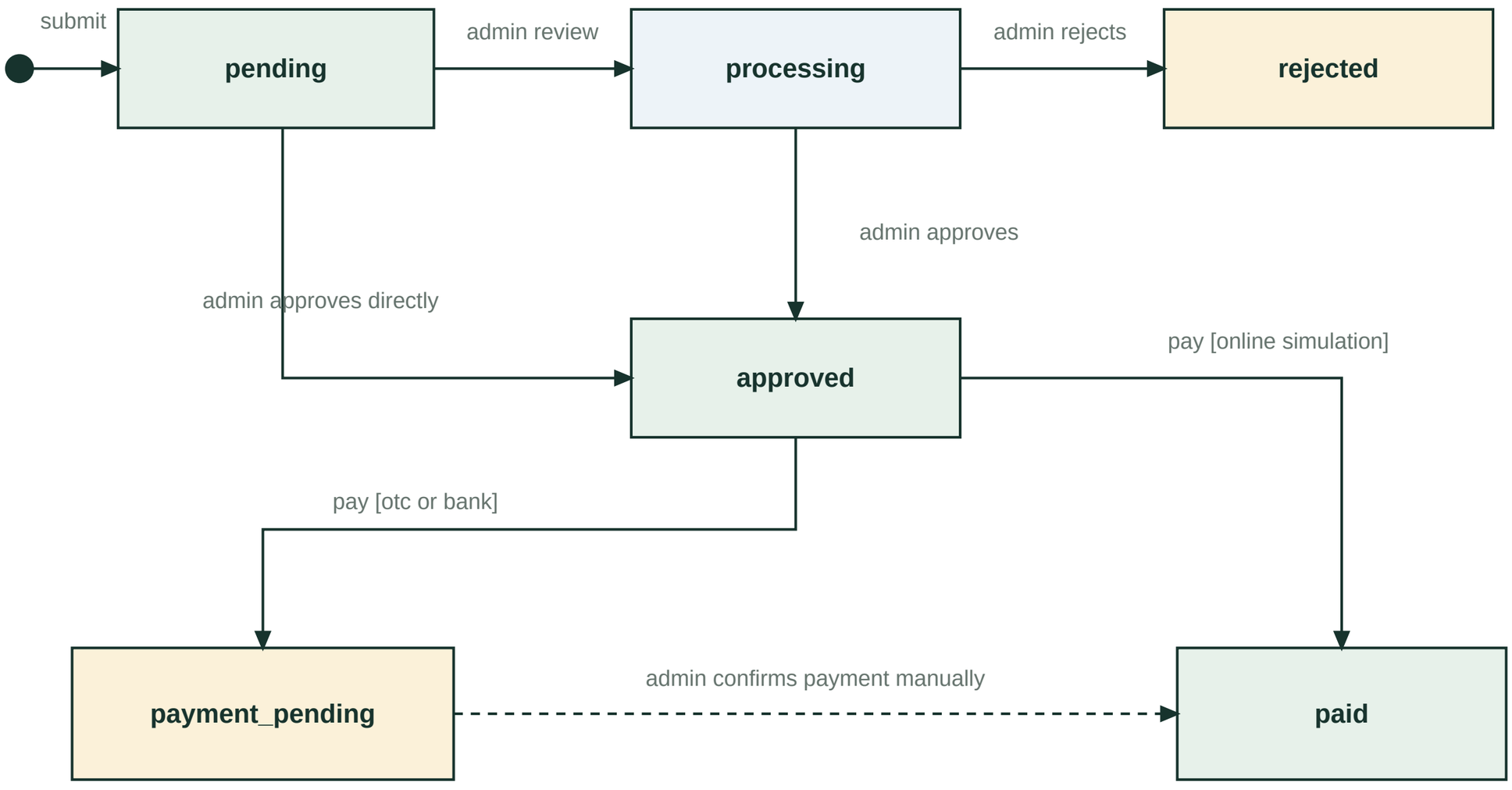


Successful submission path. The browser subsequently follows the redirect with a separate GET, which reads the saved row and renders its confirmation page.

State Machine Diagram

Behavioral diagram | Supplementary reference view

Model the normal request status lifecycle and payment branches.



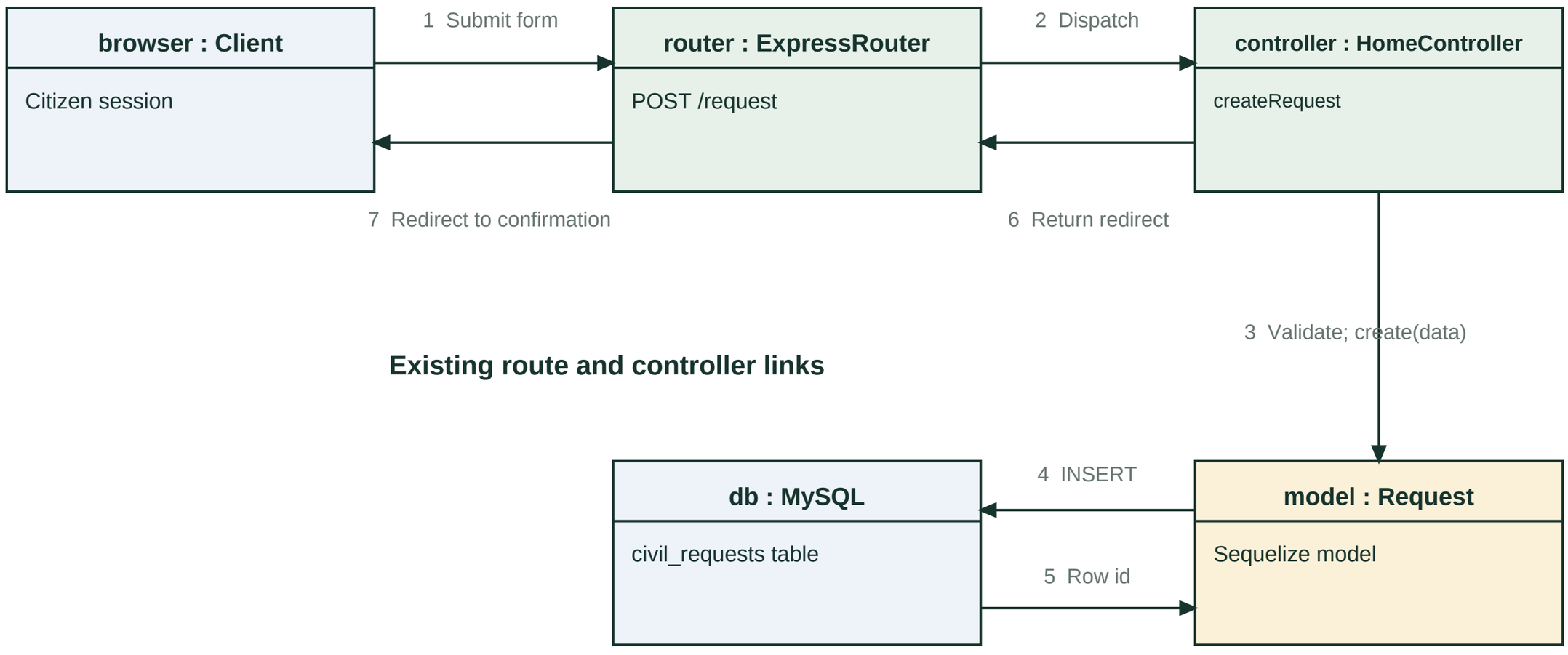
No automated bank reconciliation or payment-provider callback exists

The lifecycle is descriptive. The current admin endpoint accepts arbitrary status strings and does not enforce every transition shown. Pending is the model default; submission writes lowercase pending.

Communication Diagram

Behavioral diagram | Supplementary reference view

Show the collaboration links for a valid request submission using numbered messages.

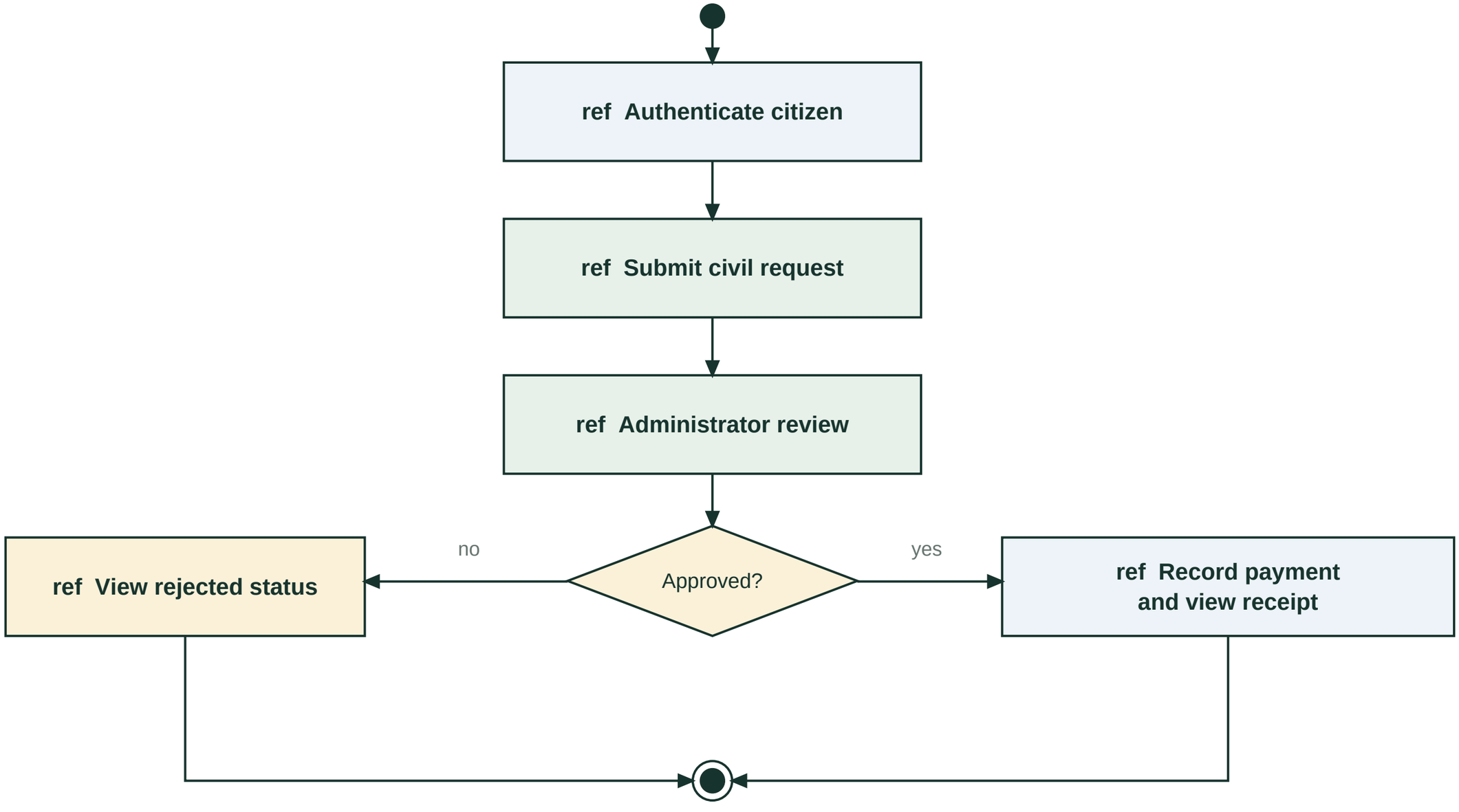


Message numbering expresses order. This is the same scenario as the sequence diagram, presented as a communication network.

Interaction Overview Diagram

Behavioral diagram | Supplementary reference view

Coordinate major interactions across the request and payment journey.



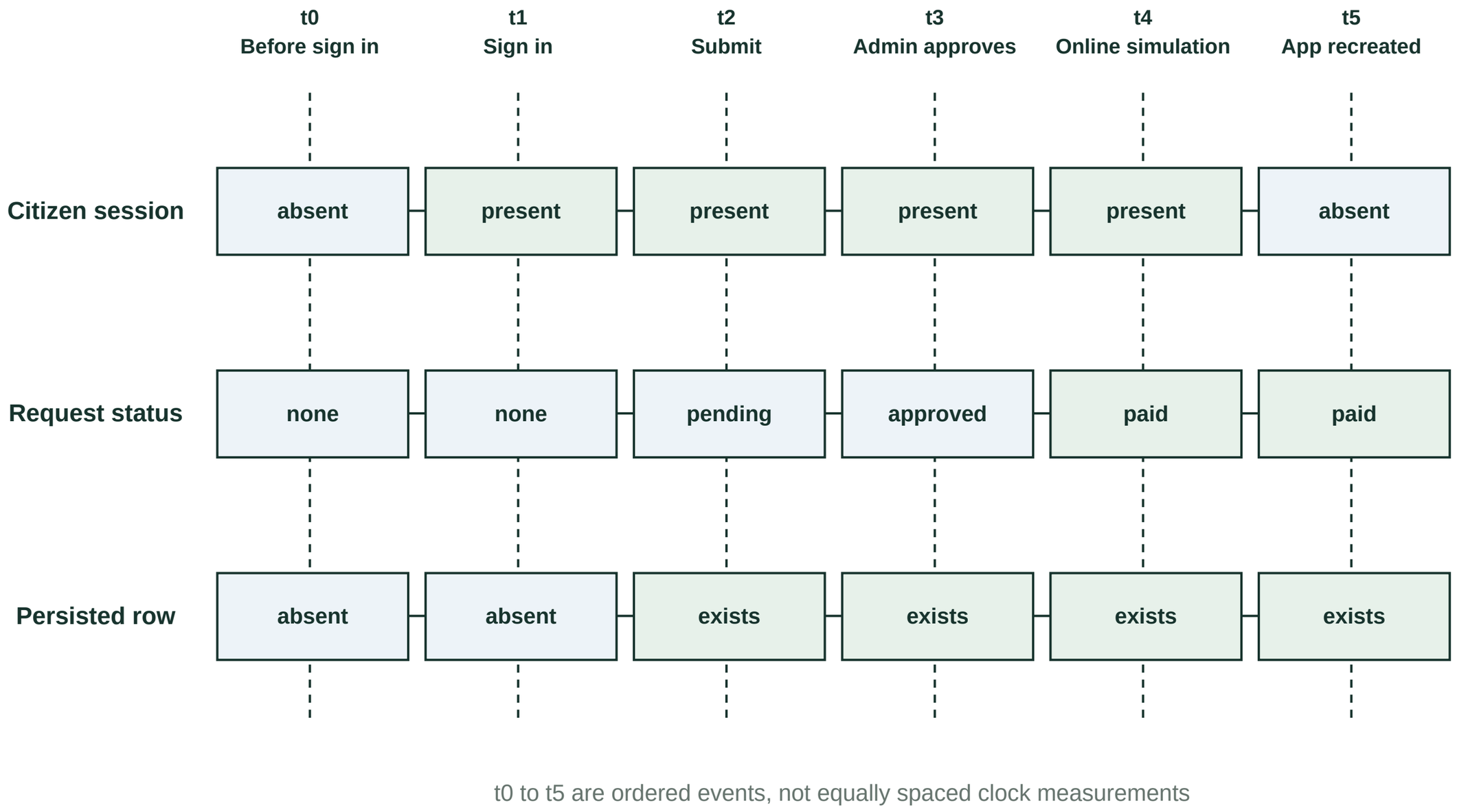
ref = interaction reference; payment branch is a prototype flow

Each ref frame denotes a separate interaction, not an automatic background task. Rejection and administrator status changes are explicit decisions.

Timing Diagram

Behavioral diagram | Supplementary reference view

Compare session, request and record state at ordered events.

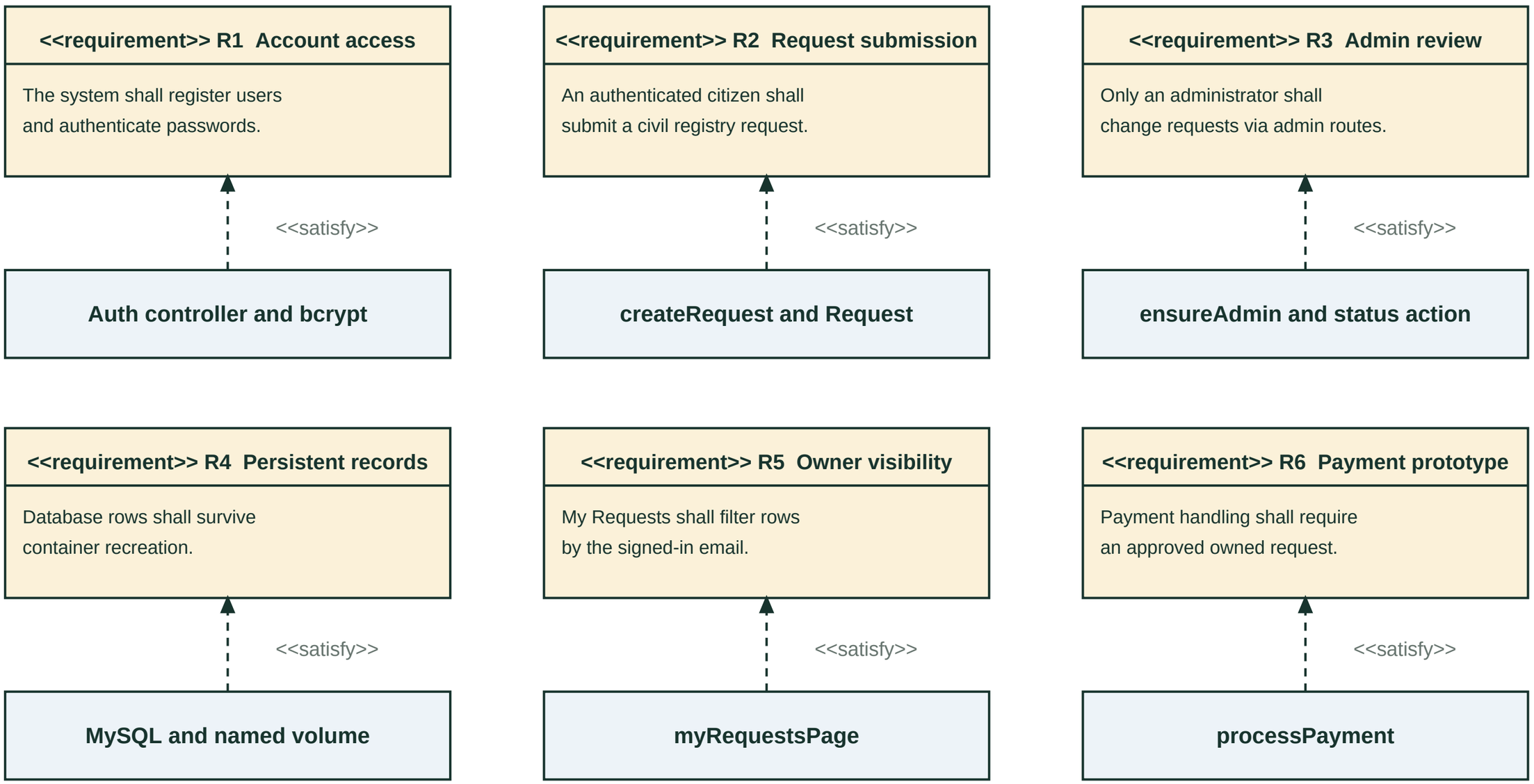


Logical event ordering only. No measured latency, deadline, real-time guarantee or external payment timing is implied. Session storage resets when the app process is recreated.

Requirement Diagram

Behavioral reference SysML diagram | Supplementary reference view

Trace system requirements to the components that satisfy them.



SysML requirement diagrams are included because the assignment reference lists them. They are requirements traceability models rather than behavioral UML diagrams.

Implementation Notes and Source Map

Use these distinctions when presenting the models

Current application

The code is a JavaScript Express application with Handlebars-compatible .xian templates. Browser pages and HTTP handlers run in one application container. There is no React service or Nginx service in compose.yaml.

Ownership and model relationships

User and Request share email values, but the schema contains no Request.userId foreign key. Attachment.requestId and Appointment.requestId reference civil_requests.id. Two source files define User; the diagram consolidates their shared persisted fields.

Appointments and supporting files

Appointment and Attachment models exist, but appointment controllers currently operate on Request rows. Several appointment update fields are absent from that model, and the attachment page does not implement a file-upload workflow. A model or route name alone is not evidence that the full feature works.

Payments and status changes

The online branch simulates success. OTC and bank branches store payment_pending with a generated reference; there is no external provider confirmation. Status is a free-form string, and the administrator action does not enforce a transition graph. The state diagram describes the intended normal flow.

Access control and page availability

Several admin request mutations and user-management routes use ensureAdmin. Protection is not consistent across every admin-named page. Some controllers reference templates that are not present; the diagrams describe module behavior and do not certify every screen as operational.

Optional reference types

The profile is proposed modeling vocabulary. Object values are fictional. Timing shows ordered events rather than measured performance. The Requirement Diagram is SysML, retained to match the assignment reference list.

Repository sources

index.js; routes/index.js; middleware/auth.js; controllers/authController.js; controllers/homeController.js; models/db.js; models/userModel.js; models/User.js; models/Request.js; models/Appointment.js; models/Attachment.js; compose.yaml; Dockerfile.

Assignment source

diagrams (1).docx, supplied by the user. Its exercise asks for one structural selection, two behavioral selections, project identification, and a data-flow explanation. All listed reference types are additionally included here at the user request.